Universidade Federal do Rio Grande do Norte

Departamento de Informática e Matemática Aplicada

DIM0548 – Engenharia de Linguagens

Grupo 2:

Caio Daniel, Flawbert Costa, Murilo Costa, Pedro Paiva e Rodrigo Ferreira.

Referências:

Livro: Concepts of Programming Languages. Robert W. Sebesta. Addison-Wesley, 11th edition, 2016.

https://github.com/winstondu/Algorithm_Papers/blob/master/Robert%20W.%20Sebesta%20Concepts%20of%20Programming%20Languages.pdf

Respostas:

1. **A frase acima é o motivo do desenvolvimento de técnicas eficientes para o parsing. Uma dessas técnicas é o parsing LR(1). Que significa a sigla LR(1)? OBS: Consultar a referência sugerida para uma resposta concisa a esta pergunta.**

   A sigla LR(1) descreve uma classe de analisadores sintáticos bottom-up. O primeiro "L" indica que a entrada é lida da esquerda para a direita (*Left-to-right*). O "R" indica que o analisador constrói uma derivação mais à direita (*Rightmost derivation*) em ordem inversa, ou seja, reduz a cadeia de entrada até chegar ao símbolo inicial da gramática. O "(1)" indica que o analisador usa um único símbolo de lookahead, isto é, olha um token à frente para decidir a próxima ação.

   Na prática, o analisador LR mantém uma pilha e uma tabela de parsing (ações *shift*, *reduce*, *accept* e *error*, mais a tabela de desvios *goto*). A cada passo, ele empilha o próximo token ou reduz um *handle* (o lado direito de uma produção no topo da pilha) ao seu não-terminal. O resultado é um parsing eficiente, de tempo linear no tamanho da entrada, que reconhece uma classe de gramáticas maior do que a dos analisadores top-down.

2. **Os algoritmos de parsing LR mais comuns são: SLR, LR(1) e LALR. Quais são as diferenças entre eles?**

   Os três algoritmos constroem o mesmo tipo de autômato de pilha e diferem na precisão com que decidem quando reduzir. O SLR (*Simple LR*) usa itens LR(0), sem lookahead, e só faz a redução $A \rightarrow \alpha$ se o próximo token pertencer ao conjunto FOLLOW(A). É o mais simples e gera as tabelas menores, mas é o menos poderoso, pois FOLLOW é uma aproximação grosseira e produz conflitos em gramáticas que não são ambíguas. O LR(1) canônico usa itens LR(1), em que cada item carrega o seu próprio símbolo de lookahead. Isso permite decisões exatas e o torna o mais poderoso dos três, mas o número de estados cresce muito, o que torna as tabelas grandes e a construção custosa.

   O LALR (*Look-Ahead LR*) é um meio-termo entre os dois. Ele constrói o autômato LR(1) e funde os estados que têm o mesmo núcleo (os mesmos itens LR(0)), unindo seus lookaheads. Com isso, tem o mesmo número de estados do SLR (tabelas compactas) e poder próximo ao do LR(1), embora a fusão possa, em casos raros, introduzir conflitos reduce/reduce que o LR(1) não teria. Por esse equilíbrio entre poder e tamanho, o LALR é o algoritmo usado por geradores como Yacc e Bison.

3. **Yacc é uma ferramenta para gerar analisadores sintáticos LALR. Qual é a sintaxe de um programa Yacc?**

   Um programa Yacc é dividido em três seções separadas por %%: declarações, regras e rotinas do usuário (declarações %% regras %% código C). A seção de declarações contém código C entre %{ e %} (includes, variáveis globais) e as declarações do Yacc: %token (tokens da linguagem), %left, %right e %nonassoc (precedência e associatividade), %union e %type (tipos dos valores semânticos) e %start (símbolo inicial). A seção de regras

contém as produções da gramática no formato não-terminal : “símbolos { ação em C } | outra alternativa;”, em que $$ é o valor do lado esquerdo e $1, $2, ... são os valores dos símbolos do lado direito. A terceira seção traz código C copiado integralmente para o arquivo gerado, normalmente a main (que chama yyparse()), a yyerror (tratamento de erros) e, se não houver Lex/Flex, a yylex. Essa seção é opcional.

```c
%{
#include <stdio.h>

int yylex(void);
void yyerror(const char *msg);
%}

%token NUM
%left '+' '-'
%left '*'

%%

expr : expr '+' expr    { printf("soma\n"); }
     | expr '-' expr    { printf("subtração\n"); }
     | expr '*' expr    { printf("multiplicação\n"); }
     | '(' expr ')'
     | NUM
     ;

%%

void yyerror(const char *msg) {
    fprintf(stderr, "Erro: %s\n", msg);
}

int main(void) {
    return yyparse();
}
```

No exemplo, a primeira seção (entre %{ e %}) traz o código C de apoio, com o #include e os protótipos de yylex e yyerror. Logo abaixo ficam as declarações do Yacc: %token NUM define o token numérico e %left define a precedência e a associatividade dos operadores (* tem precedência maior que + e -, por ser declarado depois, e todos associam à esquerda). Na segunda seção, após o primeiro %%, está a gramática: a regra expr tem cinco alternativas (soma, subtração, multiplicação, expressão entre parênteses e número), e as ações entre chaves são executadas quando a produção é reduzida, nesse caso apenas imprimindo qual operação foi reconhecida. Na terceira seção, após o segundo %%, estão a yyerror, que exibe mensagens de erro sintático, e a main, que chama yyparse() para iniciar a análise.

4. **Implementar o parser LALR para a sua linguagem, usando Yacc.**

   A implementação está salva no seguinte repositório (detalhes descritos no README.md):

   Repositório github da implementação

5. **Qual é o resultado obtido pelo seu parser para os programas de teste?**

Segue abaixo execução do testes para código ACEITO:

```
=== tests/valido/01_id_simples.txt
--- entrada:
a

--- saída:
reduz r1: E -> id        [a]
ACEITO: entrada reconhecida pela gramática
--- código de saída: 0

=== tests/valido/02_id_chamada.txt
--- entrada:
a(b)

--- saída:
reduz r1: E -> id        [b]
reduz r2: E -> id ( E )  [a ( b )]
ACEITO: entrada reconhecida pela gramática
--- código de saída: 0

=== tests/valido/03_soma.txt
--- entrada:
a+b

--- saída:
reduz r1: E -> id        [a]
reduz r3: E -> E + id    [a + b]
ACEITO: entrada reconhecida pela gramática
--- código de saída: 0

=== tests/valido/04_aninhado.txt
--- entrada:
a(b+c)

--- saída:
reduz r1: E -> id        [b]
reduz r3: E -> E + id    [b + c]
reduz r2: E -> id ( E )  [a ( b )]
ACEITO: entrada reconhecida pela gramática
--- código de saída: 0
```

Segue abaixo execução do testes para código REJEITADO:

```
=== tests/invalido/01_erro_lexico.txt
--- entrada:
a#b

--- saída:
1: caractere inválido perto de '#'
1: syntax error perto de '#'
reduz r1: E -> id        [a]
REJEITADO: entrada não pertence à linguagem
--- código de saída: 1

=== tests/invalido/02_erro_sintatico_parenteses.txt
--- entrada:
a(b

--- saída:
2: syntax error perto de ''
reduz r1: E -> id        [b]
REJEITADO: entrada não pertence à linguagem
--- código de saída: 1

=== tests/invalido/03_erro_sintatico_soma.txt
--- entrada:
a+

--- saída:
2: syntax error perto de ''
reduz r1: E -> id        [a]
REJEITADO: entrada não pertence à linguagem
--- código de saída: 1
```

Todos os 7 programas de teste tiveram o resultado esperado. Os 4 válidos (`a`, `a(b)`, `a+b` e `a(b+c)`) foram aceitos com código de saída 0, e as reduções impressas mostram a ordem bottom-up (em `a(b+c)`, o parser reduz `b`, depois `b + c` e por fim `a ( b )`). Os 3 inválidos foram rejeitados com código de saída 1: `a#b` por erro léxico (caractere inválido) e `a(b` e `a+` por erro sintático (parêntese não fechado e operador sem operando), detectados no fim da entrada. A mensagem de erro aparece antes das reduções só por causa do buffer de saída ao redirecionar, pois as reduções acontecem antes do erro.